
CS 170 Midterm 1

Write in the following boxes clearly and then double check.

Name :

SID :

Exam Room :

- ☐ Wheeler 150
☐ Other (Specify):

Name of student to your left :

Name of student to your right :

- **The exam will last 110 minutes.**
- The exam has 10 questions with a total of 116 points. You may be eligible to receive partial credit for your proof even if your algorithm is only partially correct or inefficient.
- Only your writing inside the answer boxes will be graded. **Anything outside the boxes will not be graded.** The last page is provided to you as a blank scratch page.
- Answer all questions. Read them carefully first. Not all parts of a problem are weighted equally.
- The problems may **not** necessarily follow the order of increasing difficulty.
- The points assigned to each problem are by no means an indication of the problem's difficulty.
- The boxes assigned to each problem are by no means an indication of the problem's difficulty.
- **If a question asks you to provide an algorithm, you do not need to provide a proof of correctness or runtime analysis unless explicitly stated.**
- Unless the problem states otherwise, you should assume constant time arithmetic on real numbers. Unless the problem states otherwise, you should assume that graphs are simple.
- If you use any algorithm from lecture and textbook as a black box, you can rely on the correctness and time/space complexity of the quoted algorithm. If you modify an algorithm from textbook or lecture, you must explain the modifications precisely and clearly, and if asked for a proof of correctness, give one from scratch or give a modified version of the textbook proof of correctness.
- Unless stated, only deterministic (non-random) algorithms will receive points.
- Assume the subparts of each question are **independent** unless otherwise stated.
- **Please write your SID on the top of each page; you will get 1 point for doing so.**
- **For multiple choice questions, please fill in the bubbles fully.**
- Good luck!

1 Asymptotics Warmup (10 points)

- (a) For each pair of functions f and g , specify whether $f = O(g)$, $g = O(f)$, or both.

f	g	$f = O(g)$	$g = O(f)$	Both
n^2	$\log_2(n^{100})$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$(\log_2 n)^n$	$n^{\log_2 n}$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$1 + 2x + 3x^2 + 4x^3$	$4 + 3x + 2x^2 + x^3$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$2^{e^{(\log_2 n)}}$	$10^{e^{(\log_{10} n)}}$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$n^{10} + 2^n$	$n^2 + 10^n$	$\bigcirc$	$\bigcirc$	$\bigcirc$

Solution:

- (a) $g = O(f)$ only. Note that $\log_2(n^{100}) = 100 \log_2(n)$, which is a logarithmic function and grows slower than n^2 .
- (b) $g = O(f)$ only. We can rewrite $(\log_2 n)^n = 2^{n \log_2 \log_2 n}$ and $n^{\log_2 n} = 2^{(\log_2 n)^2}$. The exponent $n \log_2 \log_2 n$ grows faster than the exponent $(\log_2 n)^2$.
- (c) Both $g = O(f)$ and $f = O(g)$. They are both $\Theta(n^3)$.
- (d) $g = O(f)$ only. We bring them to the same base again, and $e^{\log_2 n}$ grows faster than $e^{\log_{10} n}$, even with a smaller leading coefficient.
- (e) $f = O(g)$. Note that $f = \Theta(2^n)$ and $g = \Theta(10^n)$.

- (b) Find, **with proof**, the asymptotic behavior of the function $T(n)$ satisfying the recurrence relation $T(n) = T\left(\frac{n}{2}\right) + 2T\left(\frac{n}{4}\right) + \Theta(n)$.

Hint: There are multiple ways to solve the problem. Here is a hint for one of them: define a new function $S(n) = T(n) + T\left(\frac{n}{2}\right)$ and substitute. Find the asymptotic value of $S(n)$ and show that $S(n)$ and $T(n)$ have the same asymptotic behavior.

Solution: The recurrence relation can be rewritten as $T(n) + T\left(\frac{n}{2}\right) = 2\left(T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right)\right) + \Theta(n)$. Substituting the relationship $S(n) = T(n) + T\left(\frac{n}{2}\right)$, this can be rewritten as $S(n) = 2S\left(\frac{n}{2}\right) + \Theta(n)$. By the master theorem, $S(n) = \Theta(n \log n)$. Since $T(n) \leq S(n) \leq 2T(n)$, it is also the case that $T(n) = \Theta(n \log n)$.

Alternate solution: This is $\Theta(n \log n)$. Define $A_i = \#$ of nodes on level i for all $i \geq 0$. We show by induction that $2^{i-1} \leq A_i \leq 2^i$ for every level $i \geq 1$. First, drawing out the tree, we see that $A_i = A_{i-1} + 2A_{i-2}$. We have that $A_1 = 1, A_2 = 3$ as base cases, and then $A_i = A_{i-1} + 2A_{i-2} \leq 2^{i-1} + 2 \cdot 2^{i-2} = 2^i$ and $A_i = A_{i-1} + 2A_{i-2} \geq 2^{i-2} + 2 \cdot 2^{i-3} = 2^{i-1}$.

We then apply the Squeeze Theorem: the work, $\sum_{i=0}^{\log_2 n} \binom{n}{2^i} A_i$, is upper bounded by

$$\sum_{i=0}^{\log_2 n} \binom{n}{2^i} 2^i = n \log_2 n$$

and lower bounded by

$$\sum_{i=0}^{\log_2 n} \binom{n}{2^i} \frac{2^i}{2} = \frac{1}{2} n \log_2 n.$$

Hence, we conclude that $T(n) = \Theta(n \log n)$.

2 True/False (6 points)

For each question, fill in True or False. Justification is not required and will not earn partial credit.

- (a) True or False: The vertex with the smallest post-number in the DFS on a DAG is necessarily a sink vertex.

☐ True ☐ False

Solution: True. This holds due to the properties of a DAG mentioned in lecture.

- (b) Consider two data streams, S and S' , each consisting of 170000 characters belonging to the set $\{A, B, C, D\}$. The relative character frequencies in each stream are shown below:

$$S : f_A = 0.7, f_B = 0.1, f_C = 0.1, f_D = 0.1,$$

$$S' : f'_A = f'_B = f'_C = f'_D = 0.25.$$

True or False: Using Huffman encodings, it takes fewer bits to encode S compared to S' .

☐ True ☐ False

Solution: True. A uniform frequency distribution is one upon which a Huffman encoding is least effective.

- (c) Given a weighted graph G , you make a new graph G' by raising the weight of each edge to the power of 5.

True or False: The set of all minimum spanning trees in G and G' are the same.

☐ True ☐ False

Solution: True. Raising all edges to the power of 5 preserves the relative ordering between edges.

- (d) True or False: Any minimum spanning tree includes all of the graph's lightest edges.

☐ True ☐ False

Solution: False. If a graph has multiple lightest edges they could form a cycle.

- (e) True or False: If a graph has a unique heaviest edge, that edge belongs to no minimum spanning tree of the graph.

☐ True ☐ False

Solution: False. There may exist a cut for which the heaviest edge is the only edge crossing it.

- (f) True or False: In any undirected graph with any given source vertex s , there exists a shortest path from s to some other vertex that includes a minimum weight edge of the graph.

☐ True ☐ False

Solution: False. Consider the triangle on vertices $\{A, B, C\}$, where the edges have weights

$$w(A, B) = 5, \quad w(A, C) = 5, \quad w(B, C) = 1.$$

Given the source $s = A$, the shortest path tree in this graph consists of the edges (A, B) and (A, C) ; the minimum-weight edge (B, C) is not included.

3 Short Answer Questions (18 points)

Write down your answers in the corresponding boxes. Justification is not required and will not earn partial credit.

- (a) In a simple graph with 10 vertices and 10 undirected edges (that is, without self-loops and with at most one edge between any pair of vertices), the minimum possible and maximum possible number of connected components are:

Minimum: , Maximum:

Solution: Minimum is 1, Maximum is 6 (K_5 and 5 singletons).

- (b) Inspired by Karatsuba's algorithm, your friend in CS 170 has found a better divide-and-conquer method for integer multiplication. In her algorithm, she expresses the multiplication of two n -digit integers as the summation of five subproblems, each involving the multiplication of two $(n/3)$ -digit integers.

- (i) Write down the recurrence relation for the runtime $T(n)$ of your friend's algorithm:

Solution: $T(n) = 5T(n/3) + O(n)$

- (ii) Write down the tightest asymptotic bound on $T(n)$ you can give.

Solution: Using Master Theorem, we get $O(n^{\log_3(5)})$.

Remark: this describes the Toom-Cook algorithm for integer multiplication!

- (c) Let $\text{Select}(A, k)$ denote the randomized QuickSelect algorithm you saw in class that finds the k^{th} smallest number in array A . Consider the execution of $\text{Select}(A, 1)$ on a list A of size n . Let p denote the number of pivots chosen by the algorithm before it terminates.

- (i) In the best case, the value of p (up to constant factors) =

- (ii) In the worst case, the value of p (up to constant factors) =

- (iii) The expected value of p (up to constant factors) =

Solution:

(i) 1

(ii) n

(iii) $\log n$

- (d) The greedy algorithm on a HornSAT instance returns the following assignment:

$$x_1 = \text{False}, x_2 = \text{True}, x_3 = \text{True}, x_4 = \text{False}, x_5 = \text{True}.$$

For each of the following clauses, indicate whether adding it will make the instance always, sometimes, or never satisfiable. Each sub-part below is independent of the other.

(i) $\overline{x_3} \vee \overline{x_4}$

☐ Always	☐ Sometimes	☐ Never
------------------------------	---------------------------------	-----------------------------

(ii) $\overline{x_3} \vee \overline{x_5}$

☐ Always	☐ Sometimes	☐ Never
------------------------------	---------------------------------	-----------------------------

(iii) $x_3 \implies x_1$

☐ Always	☐ Sometimes	☐ Never
------------------------------	---------------------------------	-----------------------------

(iv) $\overline{x_5}$

☐ Always	☐ Sometimes	☐ Never
------------------------------	---------------------------------	-----------------------------

Solution:

- (a) Always satisfiable.
- (b) Never satisfiable.
- (c) Sometimes satisfiable.
- (d) Never satisfiable.

- (e) Let G be an unweighted, undirected graph with vertex set $\{A, B, C, D, E, F\}$. Suppose we run a Single Source Shortest Path algorithm on the graph G with source vertex A and it computes the following distances:

$$[A : 0], [B : 1], [C : 1], [D : 2], [E : 3], [F : 2].$$

Answer the following questions about this graph:

- (i) Identify an edge that must exist in G .

Solution: (A, B) or (A, C) . Any distance 1 vertex must be directly connected to the source A .

- (ii) True or False: There cannot be an edge (C, E) in G .

☐ True	☐ False
----------------------------	-----------------------------

Solution: True. Otherwise $d(A, E) \leq d(A, C) + 1 = 2$.

- (iii) Identify a smallest set of vertices that is guaranteed to disconnect the graph upon removal.

Solution: $\{D, F\}$. There is no edge from A to E when removing these vertices, as otherwise E 's distance label would be 1. Similarly, there are no edges from B or C to E , otherwise E 's distance would be 2.

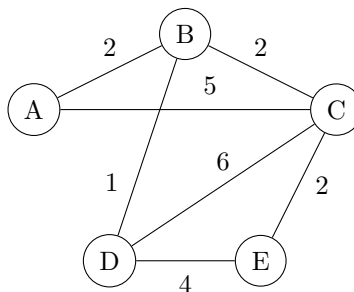
Alternate Solution: $\{B, C\}$ similarly disconnects the graph.

- (f) Consider a weighted graph G with n vertices and m edges whose edge weights are all either 1 or 2. Then, the single source shortest paths from a given node can be computed in time $O\left(\boxed{}\right)$. Express your answer in terms of n and m , and provide the tightest bound possible. Briefly justify your answer below:

Solution: $n + m$. This can be achieved by creating an unweighted graph G' that copies G but splits edges of length 2 in two by adding a dummy vertex (subdivision), and running BFS on G' .

4 Shortest Paths (10 points)

Consider the following undirected graph.



- (a) Demonstrate the execution of Dijkstra's algorithm starting from the source vertex A on the graph above. To do this, fill in the distance array, representing each node's current distance from node A at the end of each iteration of the algorithm. In each iteration, circle the node that was visited in that step. The first row has been filled out for you.

	A	B	C	D	E
dist ₁	0	2	5	∞	∞
dist ₂	—	—	—	—	—
dist ₃	—	—	—	—	—
dist ₄	—	—	—	—	—
dist ₅	—	—	—	—	—

Solution:

	A	B	C	D	E
dist ₁	0	2	5	∞	∞
dist ₂	0	2	4	3	∞
dist ₃	0	2	4	3	7
dist ₄	0	2	4	3	6
dist ₅	0	2	4	3	6

- (b) For each vertex v , write down the shortest path from vertex A to v , as determined by your execution of Dijkstra's algorithm. Write the path as a sequence of nodes visited on the path starting with A and ending with v . For example, a valid (but not necessarily shortest) path from vertex A to D is $A \rightarrow C \rightarrow B \rightarrow D$.

A to B :		A to D :	
A to C :		A to E :	

Solution:

The shortest paths from vertex A to each vertex are given below:

- A to B : $A \rightarrow B$
- A to C : $A \rightarrow B \rightarrow C$
- A to D : $A \rightarrow B \rightarrow D$
- A to E : $A \rightarrow B \rightarrow C \rightarrow E$

- (c) You want to delete the maximum number of edges possible from a graph, while ensuring that the lengths of the shortest paths from the source A to any of the vertices does not increase.

Formally, you will design an algorithm **ConservativeDelete**($G = (V, E), \{w_e\}_{e \in E}, A$) that takes as input a graph G , edge lengths w_e , and the source vertex A . Your algorithm must return a set $R \subseteq E$ of edges to be removed, such that (1) $|R|$ is as large as possible and (2) for any vertex v , the shortest $A \rightarrow v$ path using the edges in $E \setminus R$ is equal to the length of the shortest path from A to v in graph G .

Answer the following two questions about the above task.

- (i) What is the maximum number of edges that can be deleted, in terms of $|V|$ and $|E|$?

Solution:

$|E| - (|V| - 1) = |E| - |V| + 1$. Note that the shortest paths tree always has $|V| - 1$ edges in it, so we can just delete all other edges in the graph.

- (ii) Write a description of **ConservativeDelete**. Make sure that it's clear in your description how you are tracking the edges that you intend to keep/remove. Your algorithm must run in time $O((|E| + |V|) \log(|V|))$ to receive full credit.

Solution:

ConservativeDelete($G = (V, E), \{w_e\}_{e \in E}, A$):

- (1) Run Dijkstra's on (G, w, A) to return the edges in the shortest paths tree $T \subset E$ as a set of edges (this can be done by backtracking via the **prev** array).
- (2) Output $R = E \setminus T$ (i.e. all edges not in the shortest paths tree).

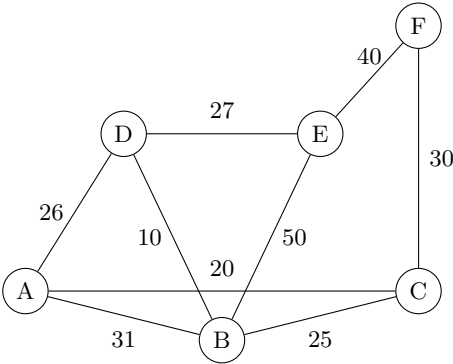
Runtime Analysis: Running Dijkstra's takes $O((|E| + |V|) \log |V|)$ time, and constructing the shortest paths tree via backtracking takes $O(|V|)$ time. Finally, constructing R takes $O(|V|)$ time, assuming the use of a hashset data structure. Thus, the overall runtime is

$$O((|E| + |V|) \log |V|) + O(|V|) + O(|V|) = O((|E| + |V|) \log |V|).$$

5 Kruskal’s MST (6 points)

Consider the undirected, weighted graph G whose vertex set is $V(G) = \{A, B, C, D, E, F\}$. Below, we list the edges along with their corresponding weights and also provide a visual representation of the graph for clarity.

- $(A, B) = 31$
- $(A, C) = 20$
- $(A, D) = 26$
- $(B, C) = 25$
- $(B, D) = 10$
- $(B, E) = 50$
- $(C, F) = 30$
- $(D, E) = 27$
- $(E, F) = 40$



Suppose we run Kruskal’s algorithm on G in order to compute the MST. For each step it takes, write down the edge Kruskal’s algorithm considers and indicate whether it adds the edge to the MST or skips it.

	Edge accessed	Added to MST? (Yes/No)
Step 1:		
Step 2:		
Step 3:		
Step 4:		
Step 5:		
Step 6:		
Step 7:		
Step 8:		
Step 9:		

Solution:
Recall that Kruskal’s algorithm does the following:

1. *Sorted edges by weight:*

$$(B, D) = 10, (A, C) = 20, (B, C) = 25, (A, D) = 26, (D, E) = 27,$$

$$(C, F) = 30, (A, B) = 31, (E, F) = 40, (B, E) = 50.$$

2. *Process edges in ascending order, adding if no cycle is formed:* Hence, we get the following:

Step 1: (B, D) , weight 10. No cycle; *add*. MST so far : $\{(B, D)\}$.

Step 2: (A, C) , weight 20. No cycle; *add*. MST so far : $\{(B, D), (A, C)\}$.

Step 3: (B, C) , weight 25. Right now, B is connected to D ; A is connected to C . Because B and C belong to different connected components, no cycle is formed; *add*. MST so far : $\{(B, D), (A, C), (B, C)\}$.

Step 4: (A, D) , weight 26. Now $\{A, B, C, D\}$ are all in one connected component via the edges already chosen. Adding (A, D) would create a cycle $(A \rightarrow C \rightarrow B \rightarrow D \rightarrow A)$. *Skip* this edge.

Step 5: (D, E) , weight 27. Vertex E is not yet in the MST, so adding (D, E) does not form a cycle; *add*. MST so far : $\{(B, D), (A, C), (B, C), (D, E)\}$.

Step 6: (C, F) , weight 30. Vertex F is not yet in the MST, so *add* (C, F) . MST so far :

$$\{(B, D), (A, C), (B, C), (D, E), (C, F)\}.$$

We have now chosen 5 edges in a graph of 6 vertices, so we have a spanning tree. All other edges would create a cycle so we should not add them.

Step 7: (A, B) , weight 31. *Skip* this edge.

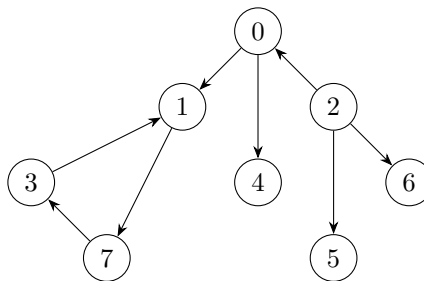
Step 8: (E, F) , weight 40. *Skip* this edge.

Step 9: (B, E) , weight 50. *Skip* this edge.

6 Pruning DFS Forest (10 points)

When running DFS, we obtain a *DFS forest* consisting of the tree(s) made up of the edges traversed by the DFS exploration procedure (tree edges). The order in which the vertices are explored, including which vertex we start at, may influence the structure of the resulting DFS forest. The possible DFS forests of a graph are defined as the different forests that can arise from running DFS with different exploration orders.

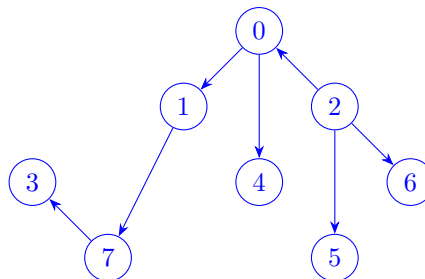
- (a) Consider the following directed graph. In this problem, we study how the number of trees in the DFS forest varies depending on the order in which the vertices are explored.



- (i) What is the minimum number of trees in a DFS forest that can result from running DFS on the above graph? Draw a DFS forest that achieves this minimum number of trees.

Minimum number of trees:

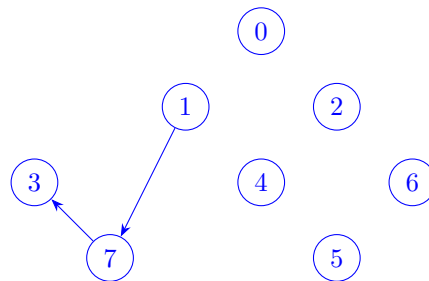
Solution: 1. See example below:



- (ii) What is the maximum number of trees in a DFS forest that can result from running DFS on the above graph? Draw a DFS forest that achieves this maximum number of trees.

Maximum number of trees:

Solution: 6. See example below:



- (b) What is the maximum number of trees in a DFS forest that can result from running DFS on a DAG with n vertices?

Solution: n . This is achieved by performing DFS in reverse topological order of vertices.

- (c) Provide an $O(n + m)$ -time algorithm that constructs a DFS forest with the maximum possible number of separate trees. The algorithm should output a list of *all* tree edges (u, v) in the DFS forest in any order.

Solution: To maximize the number of trees, we want to run DFS starting with the sink meta-nodes. To achieve this, we run Kosaraju's algorithm with the modification that we keep track of the tree edges traversed during the 2nd DFS. We then output this set of tree edges as our answer.

spies. Then, we can recursively apply this strategy to the remaining (at most) $n/2$ players. That is, we repeat this procedure until there are exactly s players left, at which point we know that they are all spies.

Alternate solution sketch: Divide the players into two groups and send each group on a mission. If a group succeeds then discard those players and if a group fails then recursively use the same strategy on that group until a group of size 1 is reached.

Runtime Analysis (not needed): After drawing out the recursion tree you'll see that at level i , at most $\min(2^i, s)$ missions can fail and those are the nodes that will branch out to the next level. There are at most $\log n$ levels. Hence, the total number of missions is bounded by,

$$\begin{aligned}\sum_{i=i}^{\log n} \min(2^i, s) &\leq \sum_{i=1}^{\log s} 2^i + \sum_{i=\log s}^{\log n} s \\ &= O(s) + s(\log n - \log s) \\ &= O(s \log(n/s))\end{aligned}$$

8 Smallest Number (15 points)

In this problem, you will design an algorithm that helps Mumble the penguin find the smallest number that can be represented with the digits of a larger number. Formally, given the base- b representation of a large number B that has exactly n digits, Mumble wants to find the smallest number A that can be obtained from B by removing all but k of its digits (and without reordering the digits).

For example, if we have $b = 5$, $B = 20334$ and $k = 3$, then some valid choices for A include 203, 034 and 204. Some invalid examples include 333, 241, 13 and 0. You should assume that the operations of storing, retrieving, and comparing base- b digits each take $O(1)$ -runtime.

- (a) Provide an algorithm `SmallestNumber( $B, k$ )` to help Mumble choose the smallest number A that can be formed by removing all but k digits from the base- b representation of B . You may assume that B is provided in its base- b representation, as a string $x_1x_2 \dots x_n$ of length n where each character x_i is an integer $x_i \in \{0, 1, \dots, b-1\}$. Your algorithm should have a runtime of $O(nk)$ (this means your runtime should not depend on the value of b) or better to receive credit.

This image shows a blank sheet of white paper designed for handwriting practice. It features a vertical solid black line on the left side, creating a margin. The rest of the page is filled with horizontal dashed black lines, providing guides for letter height and placement. There are no other markings or text on the page.

- (b) Analyze the runtime of your algorithm. You will receive credit only if your algorithm's runtime is indeed $O(nk)$.

This image shows a blank sheet of white paper designed for handwriting practice. It features ten horizontal dashed lines spaced evenly across the page. A single solid vertical line runs down the left side, creating a narrow margin. The rest of the page is open space between the dashed lines, intended for writing practice.

Solution: The main idea is to greedily choose the smallest digit in B such that you still have enough remaining digits to construct A . We provide pseudocode below.

Algorithm 1 SmallestNumber(B, k)

```

 $A \leftarrow \emptyset$ 
 $\text{start} \leftarrow 1$ 
 $\text{end} \leftarrow n - k + 1$ 
for  $i = 1, 2, \dots, k$  do
     $x \leftarrow \text{smallest digit from } B[\text{start} \dots \text{end}]$ 
     $i \leftarrow \text{index of } x \text{ in } B$ 
     $A \leftarrow A + \{x\}$ 
     $\text{start} \leftarrow i + 1$ 
     $\text{end} \leftarrow \text{end} + 1$ 
return  $A$ 

```

Runtime Analysis: In each of the k iterations of the algorithm, we check through at most n elements in B . Hence, the overall runtime is $O(nk)$, as desired.

Solution: The constructed graph $G' = (V', E')$ has $11n = O(n)$ vertices and $11m + 10|B| \leq 11m + 10n = O(m + n)$ edges. Hence, the overall runtime is $O(|V'| + |E'|) = O(n + m)$.

- (b) Now we want to do a round trip from New York to San Francisco, then back to New York in the fewest possible days while still going to at least 10 baseball games. Provide an $O(n + m)$ time algorithm that solves this problem.

[illegible]

Solution: For each $i \in \{0, \dots, 10\}$, do the following:

- (1) Use the procedure in part (a) to compute the shortest path distance from New York to San Francisco while going to at least i baseball games. Denote this distance as $d_{NYC \rightarrow SFO}^{(i)}$.
- (2) Use the procedure in part (a) to compute the shortest path distance from San Francisco to New York while going to at least $10 - i$ baseball games. Denote this distance as $d_{SFO \rightarrow NYC}^{(i)}$.
- (3) Compute $d^{(i)} = d_{NYC \rightarrow SFO}^{(i)} + d_{SFO \rightarrow NYC}^{(i)}$.

Finally, return the minimum round-trip distances computed in step (3) across all iterations of

the for loop above. Formally, we return

$$\min_{i \in \{0, \dots, 10\}} d^{(i)}.$$

Runtime Analysis (not needed): computing all of the $d^{(i)}$ takes $O(11 \cdot 2 \cdot (m+n)) = O(m+n)$, as we call the procedure in part (a) twice in each of 11 iterations. Then, computing the minimum of all the $d^{(i)}$ takes $O(10) = O(1)$ time. Hence, the overall runtime is $O(m+n) + O(1) = O(m+n)$.

Alternate Solution: We create graph G'' by creating 22 copies of G , denoted by $\{G_{i,T} = (V_{i,T}, E_{i,T})\}_{i=0}^{10} \cup \{G_{i,F} = (V_{i,F}, E_{i,F})\}_{i=0}^{10}$, where T means that San Francisco has been visited and F means it hasn't. Most edges are the same for each of $\{G_{i,T} = (V_{i,T}, E_{i,T})\}_{i=0}^{10}$ and $\{G_{i,F} = (V_{i,F}, E_{i,F})\}_{i=0}^{10}$ individually as part 9ai, except that all edges that go to San Francisco go from the F to the T copies. If a is New York, run from $a_{0,F}$ to $a_{10,T}$.

10 The Representative (15 points)

In this problem, you will design an algorithm that helps a group of penguins find a *representative* penguin, who is of somewhat average height and weight. The representative is a penguin such that the total weight of all penguins strictly shorter than it, and the total weight of all penguins strictly taller than it, are both at most half the total weight of all penguins.

Formally, suppose that n penguins with distinct heights $h_0, \dots, h_{n-1}$ have gathered in an array A , in no particular order. The i th penguin's weight is a positive value w_i . The total weight of all penguins in the array is $W = w_0 + \dots + w_{n-1}$. You will design an algorithm $\text{FindRep}(A, W_{\text{short}}, W_{\text{tall}})$ that returns a representative p (if one exists) that satisfies the following criteria:

$$\overbrace{\sum_{i:h_i < h_p} w_i}^{\text{Total weight of penguins shorter than } p} \leq W_{\text{short}} \quad \text{and} \quad \overbrace{\sum_{i:h_i > h_p} w_i}^{\text{Total weight of penguins taller than } p} \leq W_{\text{tall}}.$$

Throughout this problem, you may use the following fact without proof: if $W_{\text{short}}, W_{\text{tall}} \geq 0$ and $W_{\text{short}} + W_{\text{tall}} \geq W$, then a representative always exists. **As a reminder, only deterministic (non-random) algorithms will receive points.**

Parts (a) and (b) of this question can be found on the next two pages.

- (a) Describe a divide-and-conquer algorithm to find a representative for values of $W_{tall} = W_{short} = W/2$. Formally, you want to compute $\text{FindRep}(A, \frac{W}{2}, \frac{W}{2})$ where W is the total weight of all n penguins. Your algorithm must run in time $O(n)$ to receive full credit.

Correct algorithms with $O(n \log n)$ runtime will receive partial credit. Algorithms with worse runtime will receive no credit. You may use the algorithm **DeterministicSelect** from class, which finds the median of n numbers (such as heights) with $O(n)$ runtime.

[illegible]

Solution: Throughout our algorithm, we will keep track of the current targets for the balance condition for both sides of the scale, T_L and T_R , both of which we initialize to $\frac{W}{2}$. Instead of sorting, we use the median height penguin p as a pivot (computed via `DeterministicSelect`).

We can then partition the penguins into the left and right sides L and R (and by an abuse of notation we'll also use these to refer to the sums of the penguin weights in these respective sides). After partitioning, we check if either side violates the balance condition, i.e. if $L > T_L$ or $R > T_R$. If neither side violates, then we output the current pivot. Otherwise, at most one side violates the balance condition, since if not we deduce that $L + R > W$, a contradiction.

We will then recurse on the unbalanced side, but there are two important caveats:

- (a) We must take care to also include the pivot in the recursive call, since the current pivot might not be the pivot in the recursive call.
- (b) We must update the targets accordingly to reflect the penguins we ignore in the recursive call: if we recurse on $L \cup \{p\}$, then we update $T_R \leftarrow T_R - R$, and if we recurse on $R \cup \{p\}$ then we update $T_L \leftarrow T_L - L$.

The base case can be handled by brute force.

- (b) Analyze the runtime of your algorithm. You will receive full credit for a correct proof if your algorithm has $O(n)$ runtime and partial credit for a correct proof if your algorithm has $O(n \log n)$ runtime.

Solution: Each level of the recursion takes $O(n)$ time, since we just need to do a constant number of linear scans to update T_L and T_R , along with $O(n)$ median finding to pick the pivot.

The key observation is that, since we picked the median height, the subproblem size (measured by the *number of penguins*) has halved. Hence the runtime recurrence is $T(n) = T(\frac{n}{2}) + O(n)$. By the Master theorem, this gives us an $O(n)$ time algorithm to find the special penguin.

Use this page for scratch work.

Use this page for scratch work.

Use this page for scratch work.

Use this page for scratch work.